



Institute of Computer Science and Digital Innovation

BIC1114 Introduction to Programming

Title: Small Business Inventory and Sales System (SBISS)

Academic Session	May - September 2026
Course Code	BIC1114 / BBA1134 / BBS2324
Course Name	Introduction to Programming / Programming for FinTech / Elementary Programming
Assessment	Group Assignment
Project Title	Small Business Inventory and Sales System (SBISS)
Weightage	30%
Submission Deadline	10 August 2026

Lecturer: Asst.Prof. Dr. Samar Ghazal

COURSEWORK SUBMISSION FORM

Student Name 1	Lim Swee En
Student ID 1	1002266874
Student Name 2	Thet Htar San
Student ID 2	1002681756
Student Name 3	Nang Hsu Myat Noe
Student ID 3	1002580862
Student Name 4	Ren YueRan
Student ID 4	1002580814
Student Name 5	Hu YiFan
Student ID 5	1002681954
Subject Name	Introduction to Programming / Programming for FinTech / Elementary Programming
Subject Code	BIC1114 / BBA1134 / BBS2324
Lecturer/Tutor	Asst. Prof. Dr. Samar Ghazal
Assignment Title	Small Business Inventory and Sales System (SBISS)
Due Date	10 August 2026
Weightage	30%

STUDENTS' DECLARATION

We declare that the submitted work is our own. We confirm that we have read and understood the university regulations concerning plagiarism, collusion, and cheating. We understand that any academic misconduct may result in penalties.

Student 1 Signature <i>lim</i>	Date: 8/17/2026
Student 2 Signature <i>htar</i>	Date: 8/17/2026
Student 3 Signature <i>ren</i>	Date: 8/17/2026
Student 4 Signature <i>hu</i>	Date: 8/17/2026
Student 5 Signature <i>hsu</i>	Date: 8/17/2026
Lecturer Acknowledgement	Date:
Remarks	

Contents

1.0 Introduction	1
1.1 Background.....	1
1.2 Purpose	1
1.3 Importance of the System	2
2.0 Project Objectives	2
3.0 System Requirements.....	3
3.1 Functional Requirements	3
3.2 Business rules	4
4.0 Program Design	4
4.1 Overall System Flow	5
4.2 Menu Design.....	5
4.3 Module Design.....	5
4.4 Main Functions	6
4.5 Data Structures.....	6
5.0 Implementation	7
5.1 Main Program and Menu Control.....	7
5.2 Product Data Structure	8
5.3 Input Validation	8
5.4 Sales Calculation	8
5.5 Stock Validation.....	9
5.6 Data Storage.....	9
5.7 Report Generation	9
6.0 Testing and Results	9
6.1 Add New Product	11
6.2 Record a Sale	11
6.3 Insufficient Stock Validation	12
6.4 Save and Reopen Data	13
6.5 Sales Calculation Verification	13
7.0 Technical or Teamwork challenges and Solutions	13
8.0 Detailed Contribution Table for All Members	15
9.0 Summary	16
9.1 Lessons Learned.....	16

9.2 Possible Future Improvements	16
10.0 References:.....	17
11.0 Appendix A: Additional Testing Screenshots	17
A.1 Duplicate Product ID Validation	17
A.2 Invalid Cost Price Validation	17
A.3 Update Product Information	18
A.4 Delete Product Cancellation	18
A.5 Search Product.....	18
A.6 Low Stock and Out-of-Stock Reports	19
A.7 Missing Data File Handling.....	20

1.0 Introduction

1.1 Background

Small businesses need accurate management of product, inventory, and sales information to support daily operations. However, many businesses still rely on manual methods, which may cause data errors, delayed updates, and inaccurate stock records. Saputra et al. (2025) highlighted that manual inventory management can lead to stock errors and inconsistencies. In this project scenario, the business requires a system to manage products, inventory levels, sales transactions, and performance across multiple categories. As transactions increase, manual management becomes less effective. The Small Business Inventory and Sales System (SBISS) is developed as a menu-driven, console-based application using Python 3. It integrates product management, inventory monitoring, sales processing, and business reporting while demonstrating the practical application of fundamental programming concepts.

1.2 Purpose

The main purpose of SBISS is to provide a structured solution for managing basic inventory and sales activities in a small business. The system supports product record management, stock monitoring, sales transaction processing, and business performance analysis to improve the organisation of business information. The system applies the simulated classroom discount and tax rules specified in the project requirements. These rules include discount levels based on transaction subtotal and a 6% simulated sales tax calculation. They are implemented to demonstrate programming logic and calculation accuracy for educational purposes only and should not be considered official Malaysian tax requirements or financial regulations. Python 3 is selected as the development language because it provides clear syntax and supports essential programming features for building small-scale applications. Python provides built-in capabilities such as data structures, file handling, and exception handling, supporting practical software development without requiring external packages (Python Software Foundation, 2026).

1.3 Importance of the System

An organised inventory and sales system can improve the accuracy and efficiency of small business operations. Instead of relying on manual records, product and sales information can be stored, updated, and analysed in a structured manner. Lim et al. (2025) found that inventory management systems can improve data accuracy and operational efficiency. SBISS provides business insights through reporting functions, including inventory summaries, low-stock reports, gross profit analysis, and best-selling product information. These reports help users understand business performance and support decision-making. Overall, SBISS provides a structured approach for applying programming concepts to a realistic business problem.

2.0 Project Objectives

The main objective of this project is to design and develop a Small Business Inventory and Sales System using Python programming concepts to support basic inventory management and sales processing activities. The project aims to provide an organised approach for managing product records, monitoring inventory levels, processing sales transactions, and generating useful business information. Specifically, this project aims to develop a menu-driven Python application that allows users to manage inventory and sales activities efficiently. The system is designed to support product creation, searching, updating, deletion, and stock replenishment across different product categories. In addition, the project focuses on implementing important business rules and validation mechanisms, including maintaining unique product identifiers and ensuring that selling prices are not lower than cost prices. The project also aims to improve system reliability by applying input validation and exception handling techniques to prevent invalid operations and ensure stable program execution. Furthermore, the system is designed to process sales transactions accurately by checking product availability, updating stock quantities, and applying the required simulated classroom discount and tax rules. Another objective is to generate useful business reports, including inventory summaries, low-stock reports, gross profit analysis, and best-selling product information. Through this project, fundamental Python programming concepts, including variables, expressions, decision structures, loops, functions, strings, lists, tuples, dictionaries, file

handling, and exception handling, are applied to organise and manage product and sales data effectively.

3.0 System Requirements

The Small Business Inventory and Sales System (SBISS) is a software solution developed to streamline basic inventory management and sales processes for small businesses. It allows users to manage product information, record sales transactions, track stock levels, generate business reports, and securely store data in JSON files.

3.1 Functional Requirements

The system provides the following main functions:

Function	Description
Add New Product	Enables users to create a new product entry including the product ID, product name, category, cost price, selling price and stock quantity.
Display Products	Shows a full list of all existing products together with their up-to-date details.
Search Product	Supports product lookup based on the product ID, product name or product category.
Update Product	Permits modification of stored information for an existing product.
Delete Product	Deletes the chosen product from the system following user confirmation.
Restock Product	Increases the stock level of a specified existing product by an input quantity.
Record Sale	Logs a sales transaction containing one or more purchased items and adjusts relevant stock quantities.
Sales History	Presents a record of all previously finished sales transactions.
Generate Reports	Produces output reports covering inventory status, stock information, sales records and profit performance.
Save and Load Data	Stores product data and sales records within JSON files and retrieves the saved data automatically upon program startup.

These functions are accessible through a menu-driven interface. The menu allows the user to select the required operation by entering a number from 1 to 11.

3.2 Business rules

A set of business rules has been put in place to ensure the system remains accurate and dependable. A single sales transaction may include one or multiple product items. The system checks each Product ID and requested quantity before an item is added to the transaction record. Inventory quantities are only deducted once the transaction is finalised successfully. Discounts are applied based on the pre-discount subtotal of a transaction, as outlined below:

Subtotal	Discount Rate
Below RM100	0%
RM100 – RM199.99	5%
RM200 – RM499.99	10%
RM500 and above	15%

A simulated 6% sales tax is calculated after discount deduction.

The final transaction amount is computed with the following formula:

Final Total=Amount after Discount+Tax

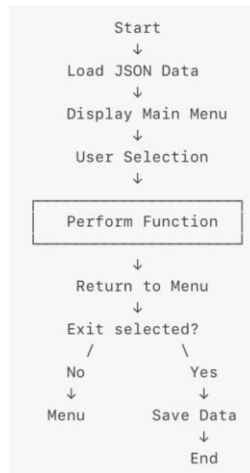
Product information and sales records are persisted in JSON files. In cases where target files are absent or contain invalid JSON syntax, the program initialises with empty product and sales record lists instead of terminating abnormally.

4.0 Program Design

SBISS adopts a modular design approach for its program architecture. This design facilitates the development, testing, readability and long-term maintenance of the application.

4.1 Overall System Flow

The general operation of the system begins by loading existing product and sales data. The process continues until the user selects Exit.



4.2 Menu Design

The system uses a menu-driven interface to provide access to all major functions. Invalid menu selections are rejected and the user is prompted to enter a valid option.

Option	Menu Function	Option	Menu Function
1	Add a new product	2	Display all products
3	Search for a product	4	Update product information
5	Delete a product	6	Restock a product
7	Record a sale	8	Display sales history
9	Generate reports	10	Save data
11	Exit		

4.3 Module Design

Module	Main Responsibility
main.py	Controls the main menu and overall program flow.
inventory.py	Handles adding, displaying, searching, updating, deleting and restocking products.
sales.py	Handles sales transactions, calculations, stock updates and receipts.
reports.py	Generates inventory, sales, stock and profit reports.
utils.py	Handles JSON data loading, saving and reusable helper functions.

4.4 Main Functions

The system uses functions to separate individual tasks.

Function	Purpose	Function	Purpose
add_product())	Adds a new product to the inventory.	display_products())	Displays all product records.
search_product())	Searches products using different criteria.	update_product())	Updates existing product information.
delete_product())	Deletes a selected product.	restock_product())	Increases the stock quantity.
record_sale())	Records a sales transaction.	calculate_discount())	Determines discount rate and amount.
calculate_tax())	Calculates the 6% simulated sales tax.	display_sales_history())	Displays completed transactions.
generate_reports())	Produces business reports.	load_data())	Loads saved JSON data.
save_data())	Saves product and sales records.		

4.5 Data Structures

The SBISS system employs three primary built-in Python data structures, namely lists, dictionaries and tuples.

```
# Store the list of inventory products
products = []

# Store the list of sales records
sales = []
```

A set is used in the category summary to store unique product categories and prevent duplicate category names.

```
195     unique_categories = set()
196
197     for product in products:
198         unique_categories.add(product["category"])
```

Dictionary: product is represented using a dictionary containing its details.

```
product = {  
    "product_id": "P001",  
    "name": "Wireless Mouse",  
    "stock": 20  
}
```

These values form a tuple and are unpacked using: `discount_rate, discount_amount = calculate_discount(subtotal)`. The tuple allows the discount rate and discount amount to be returned together from one function.

5.0 Implementation

The Small Business Inventory and Sales System (SBISS) was implemented using Python with a modular structure. The system is divided into five main modules: `main.py`, `inventory.py`, `sales.py`, `reports.py`, and `utils.py`. Each module performs a specific function, which makes the program easier to understand, maintain and test.

5.1 Main Program and Menu Control

The `main.py` module acts as the main entry point of the system. It loads existing product and sales data, displays the main menu, and calls the appropriate function according to the user's selection.

```
while True:  
    display_menu()  
    choice = input("Enter your choice (1-11): ").strip()
```

The main program uses a while loop to continuously display the menu until the user chooses to exit. User selections are handled using conditional statements.

5.2 Product Data Structure

```
new_product = {  
    "product_id": product_id,  
    "name": name,  
    "category": category,  
    "cost_price": round(cost_price, 2),  
    "selling_price": round(selling_price, 2),  
    "stock": stock,  
    "units_sold": 0  
}
```

Each product is stored as a dictionary. Multiple product dictionaries are kept inside the products list.

5.3 Input Validation

```
while True:  
    product_id = input("Enter Product ID: ").strip()  
    if product_id == "":  
        print("Error: Product ID cannot be empty.")  
    elif find_product(products, product_id) is not None:  
        print("Error: This Product ID already exists.")  
    else:  
        break
```

Validation is used to prevent duplicated Product IDs and invalid product information from being stored.

5.4 Sales Calculation

```
discount_amount = round(subtotal * rate, 2)  
return rate, discount_amount
```

The system calculates the discount based on the transaction subtotal. The function returns the discount rate and discount amount as a tuple.

```
99      return round(amount * 0.06, 2)
```

A simulated 6% sales tax is calculated after the discount.

5.5 Stock Validation

```
if quantity > available_stock:
    print(
        f" X Only {available_stock} unit(s) of "
        f"'{product['name']}' available. Try again.\n"
    )
    continue
```

The requested sales quantity is checked against available stock to prevent the inventory from becoming negative.

5.6 Data Storage

```
51      json.dump(products, file, indent=4)
```

Product and sales records are saved into JSON files so that the information remains available after the program is closed.

```
27      except (FileNotFoundError, json.JSONDecodeError):
28          products = []
```

This allows the program to continue running even when a JSON file is missing or invalid.

5.7 Report Generation

```
31      total_stock = sum(product["stock"] for product in products)
```

The report module uses Python built-in functions such as `sum()` to calculate inventory information.

6.0 Testing and Results

Test Table:

The system was tested using twelve test cases covering product management, input validation, sales processing, reporting, file handling and data persistence. Table 1 summarises the test inputs, expected results, actual results and test status. All test cases were successfully passed.

Test Case	Function	Test Input	Expected Result	Actual Result	Status
TC1	Add New Product	Add P001 with valid details	Product added successfully	Product P001 added	Pass
TC2	Duplicate Product ID	Enter existing ID P001	Duplicate ID rejected	Error message displayed	Pass
TC3	Invalid Cost Price	Enter abc as cost price	Invalid input rejected	Error message displayed	Pass
TC4	Update Product	Update P001 product name	Product information updated	Product updated successfully	Pass
TC5	Delete Product Cancellation	Select delete, enter N	Product not deleted	Deletion cancelled	Pass
TC6	Record Sale	Sell P001 $\times$ 2	Receipt generated correctly	Sale recorded successfully	Pass
TC7	Insufficient Stock	Sell more than available stock	Sale quantity rejected	Available-stock warning displayed	Pass
TC8	Search Product	Search P001 and P999	P001 found; P999 not found	Correct search results displayed	Pass
TC9	Reports	Threshold = 5	Low/out-of-stock items shown	Correct items displayed	Pass

TC10	Missing Data File	Temporarily rename JSON files	Program starts without crashing	Main menu displayed normally	Pass
TC11	Save & Reopen	Save product and restart	Saved product still exists	Product retained after restart	Pass
TC12	Calculation	Perform test sale	Discount, tax, total and profit correct	Calculations correct	Pass

6.1 Add New Product

The system successfully added product P001 to the inventory, as shown in Figure 1.

```

SMALL BUSINESS INVENTORY AND SALES SYSTEM
=====
1. Add a new product
2. Display all products
3. Search for a product
4. Update product information
5. Delete a product
6. Restock a product
7. Record a sale
8. Display sales history
9. Generate reports
10. Save data
11. Exit
=====
Enter your choice (1-11): 1

--- Add New Product ---
Enter Product ID: P001
Enter Product Name: Wireless Mouse
Enter Category: Accessories
Enter Cost Price (RM): 25
Enter Selling Price (RM): 39.90
Enter Initial Stock Quantity: 20
Success: Product P001 added.

```

Figure 1: Add New Product Function Output

6.2 Record a Sale

The system successfully recorded the sales transaction, generated a unique Sale ID, and displayed the receipt with the calculated transaction details.

```

PS C:\Users\User\OneDrive\Desktop\vscode\SBISS_Project>
10. Save data
11. Exit
=====
Enter your choice (1-11): 7

--- New Sale ---
Enter product ID and quantity for each item.
Type 'done' as the Product ID when you are finished.

Product ID (or 'done' to finish): P001
Quantity for 'Wireless Mouse Pro': 2
✓ Added 2 x Wireless Mouse Pro = RM79.80

Product ID (or 'done' to finish): done

=====
RECEIPT
=====
Sale ID: SALE-B9308D
Date/Time: 2026-08-04 00:24:49
-----
Item          Qty    Unit    Total
Wireless Mouse Pro    2    39.90    79.80
-----
Subtotal:                      RM 79.80
Discount (0%):                  -RM 0.00
Tax (6%):                       RM 4.79
TOTAL:                          RM 84.59
=====
(Internal) Gross Profit: RM29.80
=====

```

Figure 2: Successful Sales Transaction and Receipt Generation

6.3 Insufficient Stock Validation

The system rejected the requested sales quantity because it exceeded the available stock and displayed the remaining stock quantity.

```

=====
SMALL BUSINESS INVENTORY AND SALES SYSTEM
=====
1. Add a new product
2. Display all products
3. Search for a product
4. Update product information
5. Delete a product
6. Restock a product
7. Record a sale
8. Display sales history
9. Generate reports
10. Save data
11. Exit
=====
Enter your choice (1-11): 7

--- New Sale ---
Enter product ID and quantity for each item.
Type 'done' as the Product ID when you are finished.

Product ID (or 'done' to finish): P001
Quantity for 'Wireless Mouse Pro': 100
X Only 18 unit(s) of 'Wireless Mouse Pro' available. Try again.

```

Figure 3: Insufficient Stock Quantity Validation

6.4 Save and Reopen Data

The saved product records remained available after the program was closed and reopened.

```
--- Product List ---
ID      Name      Category    Cost  Price  Stock  Sold
-----
p001    Wireless Mouse Pro  Accessories  25.00  39.90   17     3
P010    USB Cable      Accessories  10.00  15.00    3     0
P020    Headphone      Accessories  20.00  30.00    0     0
P030    Test Item      Accessories  10.00  15.00    5     0
```

Figure 4: Saved Product Retained After Program Restart

6.5 Sales Calculation Verification

The system correctly calculated the subtotal, discount, tax, final total, and gross profit for the sales transaction.

```
=====
                        RECEIPT
=====
Sale ID:   SALE-EA15B7
Date/Time: 2026-08-04 01:13:53
-----
Item      Qty   Unit   Total
Keyboard    2   70.00  140.00
-----
Subtotal:                      RM 140.00
Discount (5%):                 -RM  7.00
Tax (6%):                      RM   7.98
TOTAL:                          RM 140.98
=====
(Internal) Gross Profit: RM33.00
=====
```

Figure 5: Sales Calculation and Receipt Verification

7.0 Technical or Teamwork challenges and Solutions

The development of the Small Business Inventory and Sales System (SBISS), a five-person group effort, posed some challenges in both technicalities and working as a team. Each team member had his own module to develop; these modules included main.py, inventory.py, sales.py, reports.py, and utils.py. Hence, good communication and coordination were required in order to ensure that all the modules work well together.

One of the major technical challenges that we had was the issue of data type inconsistency. When developing the program, there have been cases whereby certain price data obtained from files or user input was saved as string type rather than numerical types such as integers or floating point numbers. The solution that we had for this challenge is through using the `utils.py` module.

Overselling and negative stock at checkout was another issue. In the event a user entered a wrong quantity of items in the middle of a transaction, it was possible for certain items to be modified before the mistake was noticed, resulting in erroneous inventory information. This was resolved by incorporating validation in the `record_sale()` function to ensure that all selected items exist in the inventory and have sufficient stock before any modifications to the inventory occur.

File operations were another issue with the system as initially implemented. It was possible for the program to crash if there was an issue with the JSON file. To address this issue, try-except error handling was implemented in `utils.py`, allowing the program to safely recover from any file errors by creating missing directories and using empty data structures when needed.

Apart from coding errors, the team experienced some coordination difficulties. Because everyone was working on different files, having function names, arguments, and imports that do not match resulted in some integration errors. Member 1 coordinated the signature of the functions and made sure that the products and sales information was passed appropriately across the modules. The other problem was that `reports.py` depends on completion of the inventory and sales modules first. In order to save time, we created sample dictionaries to allow independent development and testing of the reporting functions.

Thus, the problems encountered by the team gave us an insight into the significance of modular programming, proper function design, effective communication, and error handling. Through testing various scenarios and solving the encountered problems, the team managed to create a reliable and user-friendly console application that fulfills the required business functions.

8.0 Detailed Contribution Table for All Members

Member Name	Module Ownership & Code Responsibilities	Assigned Report Section	Contribution %
Lim Swee En	main.py — Program control loop, main menu (options 1–11), module integration, data loading on startup, manual/auto-saving on exit.	Section 5 & 6: Implementation and Testing	20%
Thet Htar San	inventory.py — Product and stock management, CRUD operations, duplicate ID checks, price/stock validation, search, and restocking.	Section 7 & 8: Challenges & Individual Contributions	20%
Ren Yue Ran	sales.py — Transaction processing, stock deduction, discount band calculations (0–15%), 6% tax, gross profit, and receipt generation.	Section 3 & 4: System Requirements & Program Design	20%
Hu Yi Fan	reports.py — Analytical reports including inventory valuation, low-stock alerts, revenue, gross profit, best-sellers, and category summaries.	Section 1 & 2: Introduction & Objectives	20%
Nang Hsu Myat Noe	utils.py — Data persistence (JSON file load/save), missing folder/file exception handling, and reusable input validation routines.	Section 9: Conclusion	20%

9.0 Summary

This project created a system for managing sales and inventory. The system is used through the console and is developed using Python. It allows users to manage products, process sales, update stock levels and generate reports. All of this is done through a menu that the user can follow. Product and sales information is stored in JSON files. This means that even if the program is turned off and then started again, the information can still be loaded and used. It also helps users keep track of sales and inventory in an organized way.

The system follows a modular design, with different modules called Inventory, Sales, Reports and Utility Modules. These modules are responsible for product management, processing sales, generating reports, data storage, input validation and product searching.

Testing included missing, empty, and damaged JSON files, as well as saving and loading records. The results showed that the system handled these situations correctly without crashing and kept the data safe.

The project achieved its intended purpose. It gave us experience in Python programming and other areas, such as designing modules that work together, handling files, storing data in JSON format, dealing with errors, checking user input, testing the system, and working as a team.

9.1 Lessons Learned

This project improved our practical Python programming and teamwork skills. We learned how to divide a project into modules and integrate different components into one system. We also strengthened our understanding of functions, loops, lists, dictionaries, JSON files, exception handling, and input validation. Testing and debugging helped improve our problem-solving skills and understanding of reliable software development.

9.2 Possible Future Improvements

The system could be improved by adding a graphical user interface and changing the data storage from JSON files to a database such as MySQL or SQLite. Other features that could be added in the future include user authentication, barcode scanning, automatic low-stock alerts, report exporting, graphical dashboards, cloud storage, and multi-user support.

10.0 References:

- Lim, H. Q., Rahmat, A. R., & Suban@Ani, S. (2025). *An inventory management system for small businesses*. *Journal of Digital System Development*, 3(1), 29–44. <https://e-journal.uum.edu.my/index.php/jdsd/article/view/26384>
- Python Software Foundation. (2026). *Python 3.14.6 documentation*. <https://docs.python.org/3/>
- Saputra, M. R., Asadel, P. A., Pratama, N. F., & Herlambang, M. A. (2025). *Implementasi sistem informasi inventory pada UMKM untuk mengurangi kesalahan stok barang*. *Jurnal ESIT (E-Bisnis, Sistem Informasi, Teknologi Informasi)*, 20(3), 164–169. <https://openjournal.unpam.ac.id/index.php/ESIT/article/view/56230>

11.0 Appendix A: Additional Testing Screenshots

From 6.0: Testing and results

A.1 Duplicate Product ID Validation

The system rejected the duplicated Product ID P001 and displayed an appropriate error message.

```
=====
SMALL BUSINESS INVENTORY AND SALES SYSTEM
=====
1. Add a new product
2. Display all products
3. Search for a product
4. Update product information
5. Delete a product
6. Restock a product
7. Record a sale
8. Display sales history
9. Generate reports
10. Save data
11. Exit
=====
Enter your choice (1-11): 1

--- Add New Product ---
Enter Product ID: P001
Error: This Product ID already exists.
```

Figure A1: Duplicate Product ID Validation

A.2 Invalid Cost Price Validation

The system rejected the non-numeric cost price input and displayed an appropriate error message.

```
Enter Product ID: P002
Enter Product Name: Keyboard
Enter Category: Accessories
Enter Cost Price (RM): abc
Error: Please enter a valid number.
```

Figure A2: Invalid Cost Price Validation

A.3 Update Product Information

The system successfully updated the selected product information and displayed the latest product details.

```
=====
SMALL BUSINESS INVENTORY AND SALES SYSTEM
=====
1. Add a new product
2. Display all products
3. Search for a product
4. Update product information
5. Delete a product
6. Restock a product
7. Record a sale
8. Display sales history
9. Generate reports
10. Save data
11. Exit
=====
Enter your choice (1-11): 2

--- Product List ---
ID      Name          Category    Cost   Price   Stock   Sold
-----
p001    Wireless Mouse Pro  Accessories 25.00  39.90   20      0
```

Figure A3: Successful Product Information Update

A.4 Delete Product Cancellation

The system cancelled the deletion when the user entered **N**, and the product remained in the inventory.

```
=====
SMALL BUSINESS INVENTORY AND SALES SYSTEM
=====
1. Add a new product
2. Display all products
3. Search for a product
4. Update product information
5. Delete a product
6. Restock a product
7. Record a sale
8. Display sales history
9. Generate reports
10. Save data
11. Exit
=====
Enter your choice (1-11): 5

--- Delete Product ---
Enter Product ID to delete: P001
Delete 'Wireless Mouse Pro'? (Y/N): N
Cancelled. Product was not deleted.
```

Figure A4: Product Deletion Cancellation

A.5 Search Product

The system successfully displayed the product details for the existing Product ID P001. When the non-existing Product ID P999 was entered, the system displayed a clear message indicating that no matching product was found.

```

--- Search Product ---
Search by:
1. Product ID
2. Product Name
3. Category
Enter your choice (1-3): 1
Enter search keyword: P001

ID      Name      Category      Cost  Price  Stock  Sold
-----
p001    Wireless Mouse Pro  Accessories    25.00  39.90   17     3

```

Figure A5.1: Successful Product Search Using Existing Product ID

```

--- Search Product ---
Search by:
1. Product ID
2. Product Name
3. Category
Enter your choice (1-3): 1
Enter search keyword: P999
No matching product found.

```

Figure A5.2: Search Result for Non-Existing Product ID

A.6 Low Stock and Out-of-Stock Reports

The system correctly displayed products with stock quantities below the selected threshold and separately identified products with zero stock.

```

===== BUSINESS REPORT =====

--- Inventory Summary ---
Total Products      : 3
Total Stock Units   : 20
Inventory Cost Value: RM 455.00

--- Low Stock Report ---
Enter stock threshold: 5
P010 | USB Cable | Stock: 3
P020 | Headphone | Stock: 0

--- Out of Stock Report ---
P020 | Headphone

--- Sales Summary ---
Completed Transactions : 2
Sales Revenue         : RM 119.70

--- Gross Profit Summary ---
Gross Profit : RM 44.70

--- Best Selling Product ---
Product Name : Wireless Mouse Pro
Product ID   : p001
Units Sold   : 3

--- Highest Value Transaction ---
Sale ID : SALE-B930BD
Total   : RM 84.59

--- Category Summary ---
Accessories: 3 product(s)

```

Figure A6: Low Stock and Out-of-Stock Reports

A.7 Missing Data File Handling

The program started normally and displayed the main menu even when the JSON data files were temporarily unavailable.

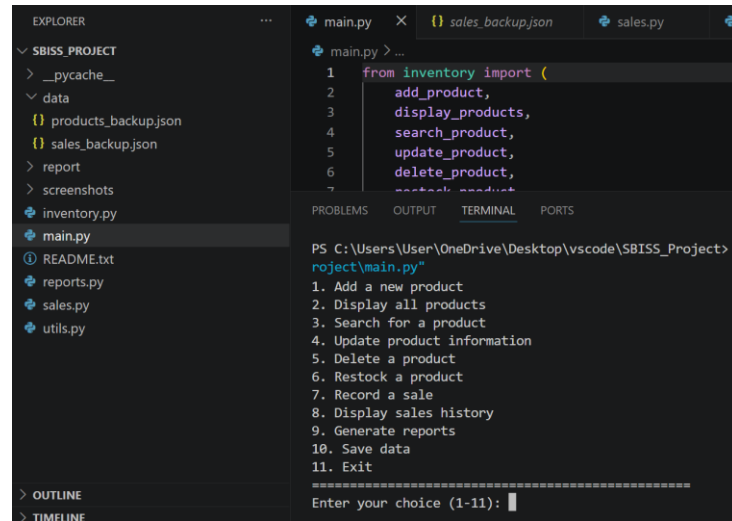


Figure A7: Program Startup Without JSON Data Files

From 4.0 Program Design

A.8 Data Structures

The SBISS system employs three primary built-in Python data structures, namely lists, dictionaries and tuples.

```
# Store the list of inventory products
products = []

# Store the list of sales records
sales = []
```

Figure A8: Code Snippet for Product and Sales List Initialization

A.9Data Structures

These values form a tuple and are unpacked using:

```
product = {  
    "product_id": "P001",  
    "name": "Wireless Mouse",  
    "stock": 20  
}
```

FigureA9: HTML-formatted Python dictionary sample for a product record